Buffer Overflow (BOF)

Corso: Cybersecurity Analyst
Modulo: W17D4 (Pratica)
Piattaforma: Epicode
Studente:Emanuele cestari
Data27/04/2026

1. Introduzione

Il Buffer Overflow è una vulnerabilità che si verifica quando un programma tenta di scrivere più dati di quanti un buffer possa contenere, causando potenzialmente corruzione della memoria.

2. Codice utilizzato

```
#include <stdio.h>
int main() {
char buffer[10];
printf("Inserisci nome: ");
scanf("%s", buffer);
printf("%s", buffer);
}
```

Codice BOF.c aperto in nano

3.Creazione file

Il codice utilizza un buffer di dimensione limitata (char buffer[10]) e la funzione scanf("%s", buffer) senza controlli sulla lunghezza dell'input. Questo può portare a un buffer overflow se l'utente inserisce una stringa più lunga della capacità del buffer.

```
  GNU nano 8.7.1
#include <stdio.h>

int main() {

    char buffer[10];

    printf("Si prega di inserire il nome utente: ");
    scanf("%s", buffer);

    printf("Nome utente inserito: %s\n", buffer);

    return 0;
}
```

COMPILAZIONE

```
┌──(kali㉿kali)-[~/Desktop]
└─$ gcc -g BOF.c -o BOF

┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Si prega di inserire il nome utente: test
Nome utente inserito: test
```

ESECUZIONE

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Si prega di inserire il nome utente: aaaaaaaaaaaaaaaaaaaaa
Nome utente inserito: aaaaaaaaaaaaaaaaaaaaa
```

ESECUZIONE DEL PROGRAMMA CON INPUT CORTO

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Si prega di inserire il nome utente: aaaaaaaaaaaaaaaaaaaaa
Nome utente inserito: aaaaaaaaaaaaaaaaaaaaa
```

ESECUZIONE CON INPUT LUNGO PER VERIFICARE OVERFLOW

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./BOF
Si prega di inserire il nome utente: aaaaaaaaaaaaaaaaaaaa
```

In fine il programma è vulnerabile anche senza crash aumentare il buffer non risolve nulla

SEZIONE FACOLTATIVA
MODIFICA AL CODICE:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define SIZE 10

int main() {
    char buffer[SIZE];

    printf("Username: ");

    scanf("%9s", buffer);  // limite corretto

    // Controllo migliorato (più robusto)
    if (strlen(buffer) ≥ SIZE - 1) {
        printf("Stringa troppo lunga (MAX: %d)\n", SIZE - 1);
        printf("Username è troncato\n");
    }

    printf("Username: %s\n", buffer);

    return 0;
}
```

Il codice è stato aggiornato con due importanti meccanismi di sicurezza. Innanzitutto, è stato impostato un limite massimo per il buffer tramite #define SIZE 10. Inoltre, la funzione scanf è stata resa sicura utilizzando lo specificatore %9s, che limita il numero massimo di caratteri letti. Infine, è stato aggiunto un controllo con strlen() per verificare se l'input raggiunge la dimensione massima del buffer. Se l'input supera questo limite, il programma segnala che la stringa è stata troncata, prevenendo così comportamenti imprevisti.

COMPILAZIONE DEL PROGRAMMA

```
┌──(kali㉿kali)-[~/Desktop]
└─$ gcc -g NoBOF.c -o NoBOF

┌──(kali㉿kali)-[~/Desktop]
└─$ ./NoBOF
Username: test
Username: test

┌──(kali㉿kali)-[~/Desktop]
└─$ ./NoBOF
Username: aaaaaaaaaaaaaaaaaaaaa
Stringa troppo lunga (MAX: 9)
Username è troncato
Username: aaaaaaaaa

┌──(kali㉿kali)-[~/Desktop]
└─$
```

Il programma è stato compilato con il compilatore gcc usando il comando: gcc -g no_bof.c -o NoBOF. Dopo la compilazione, è stato eseguito con input di diverse lunghezze, sia brevi che molto lunghi.

CONCLUSIONE.

A differenza della versione vulnerabile, il programma modificato gestisce in modo appropriato gli input troppo lunghi. Quando l'utente inserisce una stringa che supera la dimensione del buffer, il programma la limita automaticamente e segnala all'utente che la stringa è troppo lunga. Questo approccio previene efficacemente gli overflow e i comportamenti anomali, dimostrando l'efficacia delle contromisure implementate.

L'implementazione di limiti espliciti sulla lettura dell'input e di controlli sulla lunghezza della stringa è una buona pratica essenziale per la sicurezza del software. Questo metodo elimina la vulnerabilità di buffer overflow presente nel programma originale, evidenziando come una valida validazione dell'input possa prevenire errori critici di memoria.

EMANUELE CESTARI